

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Mern Stack

Practical –IX

Roll No.: T010	Name: sagar kandari
Class: TYIT	Batch: 1
Date of Assignment: 30/09/2026	Date/Time of Submission: 30/09/2026

Databse mob=ngo db code

db.users.insertMany([

```
{
name: sagar kandari",
email: sagarkandari@gmail.com",
city: "Mumbai"
}{
name: "Sagar Kandari",
email: sagar kandari@gmail.com",
city: "Pune"
},
{
name: "sagar kandari",
email: "sagar kandari@gmail.com",
city: "mumbai"
},
])
>db.users.find()
```

```
server .js
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
const PORT = 3000;
```

```
// Middleware
app.use(cors());
app.use(express.json());

// MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/UserDB") .then(() => {
  console.log("MongoDB Connected Successfully"); })
.catch((error) => {
  console.log(
    "MongoDB Connection Failed"); console.log(error);
  });

// GET - Retrieve Users
app.get("/users", async (req, res) => {
  try {
    const users = await mongoose.connection.db
      .collection("users")
      .find()
      .toArray();
    res.json(users);
  }
  catch (error) {
    res.status(500).json({message: "Error fetching users"}); }
});

// POST - Add User
app.post("/users", async (req, res) => {
  try {
    const user = req.body;

    await mongoose.connection.db
      .collection("users")
      .insertOne(user);
    res.json({message: "User added successfully"}); }
  catch (error) {
    res.status(500).json({message: "Error adding user"}); }
  });

// PUT - Update User
```

```

app.put("/users/:id", async (req, res) => {
  try {
    const id = new mongoose.Types.ObjectId(req.params.id); const updatedUser =
    req.body;

    await mongoose.connection.db
    .collection("users")
    .updateOne({ _id: id },
      {
        $set: {
          name: updatedUser.name,
          email: updatedUser.email,
          city: updatedUser.city
        }
      }
    );
    res.json({message: "User updated successfully"}); }
    catch (error) {
      res.status(500).json({message: "Error updating user"}); }
    });

    // DELETE - Delete User
    app.delete("/users/:id", async (req, res) => { try {
      const id = new mongoose.Types.ObjectId(req.params.id);

      await mongoose.connection.db
      .collection("users")
      .deleteOne({
        _id: id
      });

      res.json({message: "User deleted successfully"}); }
      catch (error) {
        res.status(500).json({message: "Error deleting user"}); }
      });

      // Start Server
      app.listen(PORT, () => {
        console.log(`Server is running o

```

```
http://localhost:${PORT}`);});
```

Create frontend

```
import { useEffect, useState } from "react";
import axios from "axios";

function App() {
  // Store all users
  const [users, setUsers] = useState([]);
  // Form data
  const [name, setName] = useState("");
    const [email, setEmail] = useState("");
    const [city, setCity] = useState("");

    // Store ID of user being edited
    const [editId, setEditId] = useState(null);

    // Get Users from MongoDB
    const getUsers = () => {
      axios.get("http://localhost:3000/users")
        .then((response) => {
          setUsers(response.data);
        })
        .catch((error) => {
          console.log(error);
        });
    };

    // Load users when page starts
    useEffect(() => {getUsers();}, []);

    // Add or Update User
    const saveUser = () => { const user = {
      name: name,
      email: email,
      city: city
    };
  };
```

```
// UPDATE existing user
    if (editId) {axios.put(`http://localhost:3000/users/${editId}`, user)

    .then((response) => {alert(response.data.message); clearForm();
    getUsers();
    })
    .catch((error) => {console.log(error);});
    }

// ADD new user
else {axios.post("http://localhost:3000/users", user)

    .then((response) => {alert(response.data.message); clearForm();
    getUsers();
    })
    .catch((error) => {console.log(error);});
    }
};

// Edit User
const editUser = (user) => {setEditId(user._id);
setName(user.name);
setEmail(user.email);
setCity(user.city);
};

// Delete User
const deleteUser = (id) => {
    if (window.confirm("Are you sure you want to delete this user?")) {
    axios.delete(`http://localhost:3000/users/${id}`)

    .then((response) => {alert(response.data.message); getUsers();
    })
    .catch((error) => {console.log(error);});
    }
};

// Clear Form
```

```
const clearForm = () => {
  setName("");
  setEmail("");
  setCity("");
  setEditId(null);
};

return (
  <div>
    <h1>User Management System</h1>
    { /* User Form */ }
    <h2>
      { editId ? "Update User" : "Add User" }
    </h2>

    <input type="text"
      placeholder="Enter Name"
      value={name}
      onChange={(e) => setName(e.target.value)}
    />

    <br /><br />
    <input type="email"
      placeholder="Enter Email"
      value={email}
      onChange={(e) => setEmail(e.target.value)} />

    <br /><br />

    <input type="text"
      placeholder="Enter City"
      value={city}
      onChange={(e) => setCity(e.target.value)} />

    <br /><br />

    <button onClick={saveUser}>
      { editId ? "Update User" : "Add User" } </button>
```

```

{editId && (<button onClick={clearForm}> Cancel
</button>
)}
<hr />

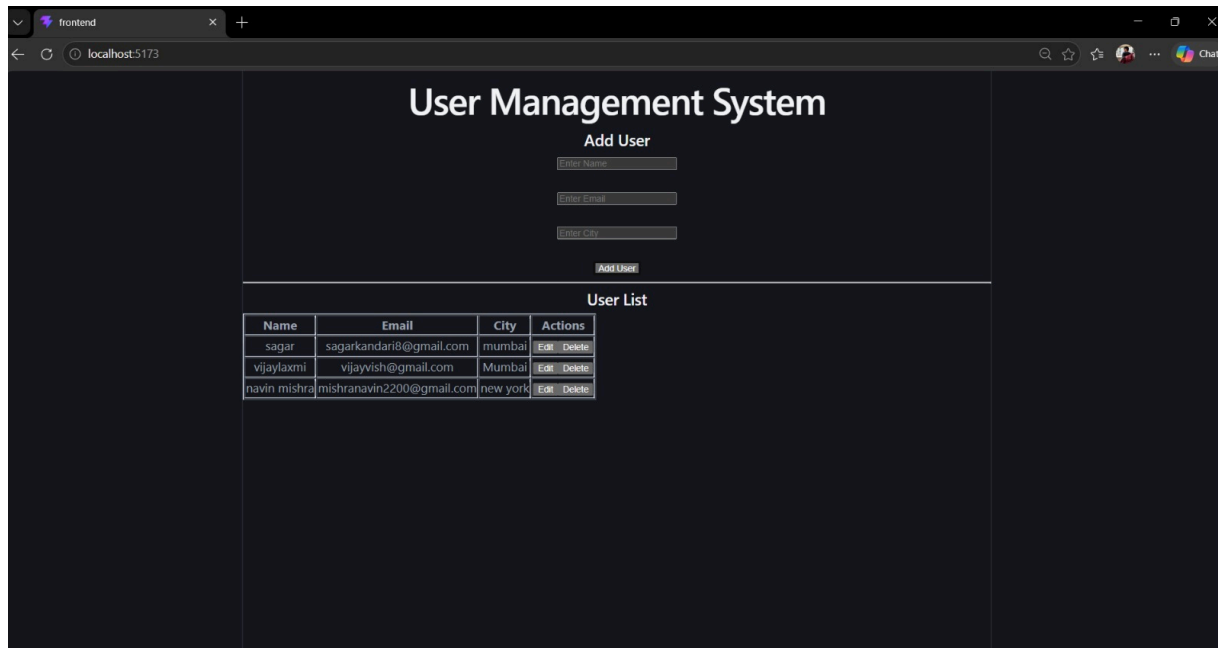
{/* Display Users */}
<h2>User List</h2>
<table border="1">
<thead>
<tr>
<th>Name</th>
<th>Email</th>
<th>City</th>
<th>Actions</th>
</tr>
</thead>

<tbody>
{users.map((user) => (
<tr key={user._id}>
<td>{user.name}</td>
<td>{user.email}</td>
<td>{user.city}</td>
<td>
<button onClick={() => editUser(user)}> Edit
</button>

<button onClick={() => deleteUser(user._id)}> Delete
</button>
</td>
</tr>
))}
</tbody>
</table>
</div>
);
}
export default App;

```

Output:-



The screenshot displays a web browser window with the title 'frontend' and the address bar showing 'localhost:5173'. The main content area features a dark-themed 'User Management System' interface. At the top, the title 'User Management System' is centered in a large, bold font. Below the title, there is an 'Add User' section with three input fields labeled 'Enter Name', 'Enter Email', and 'Enter City', each followed by a small 'Add User' button. Below this section, there is a 'User List' section containing a table with three columns: 'Name', 'Email', and 'City'. The table lists three users: 'sagar' (sagarkandari8@gmail.com, Mumbai), 'vijaylaxmi' (vijayvish@gmail.com, Mumbai), and 'navin mishra' (mishranavin2200@gmail.com, new york). Each row has an 'Actions' column with 'Edit' and 'Delete' buttons.

User Management System

Add User

Enter Name

Enter Email

Enter City

Add User

User List

Name	Email	City	Actions
sagar	sagarkandari8@gmail.com	Mumbai	Edit Delete
vijaylaxmi	vijayvish@gmail.com	Mumbai	Edit Delete
navin mishra	mishranavin2200@gmail.com	new york	Edit Delete